

CSA1713 – Artificial Intelligence

Name:- B. Deepthi reddy

Reg No:- 192312288

Slot – C

7. MISSIONARIES AND CANNIBALS PROBLEM

Code:

```
print("B. Deepthi Reddy")
print(192312288)
print("Missionaries and Cannibals Problem")
from collections import deque
q = deque([((3, 3, 0), [])])
moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]
seen = set()
while q:
    (m, c, b), path = q.popleft()
    if (m, c, b) in seen:
        continue
    seen.add((m, c, b))
    if (m, c, b) == (0, 0, 1):
        print("Solution Path:")
        print(path + [(m, c, b)])
        break
```

for x, y in moves:

if b == 0:

nm = m - x

nc = c - y

else:

nm = m + x

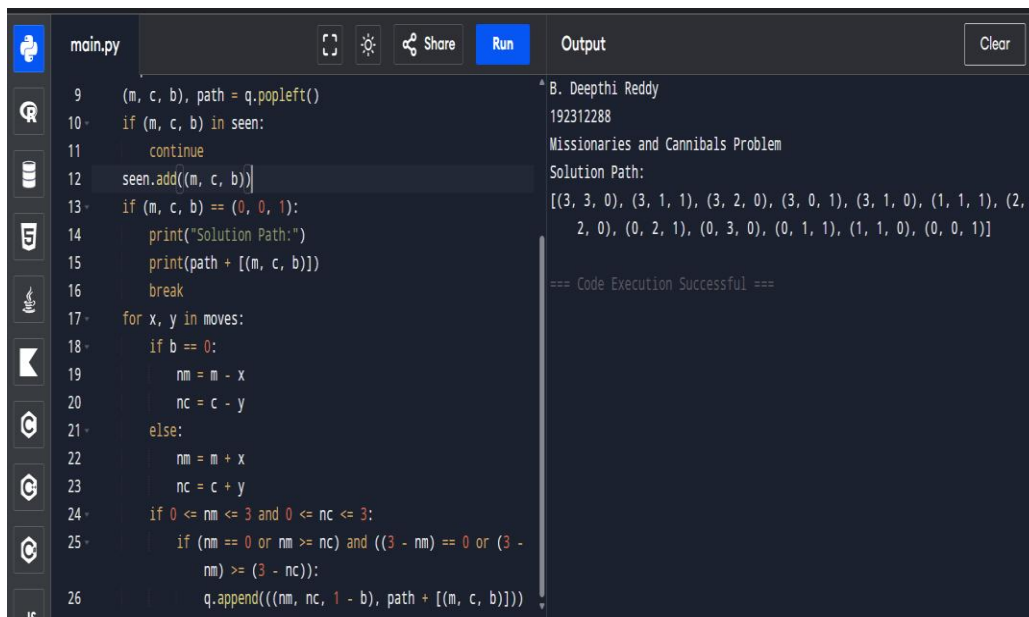
nc = c + y

if 0 <= nm <= 3 and 0 <= nc <= 3:

if (nm == 0 or nm >= nc) and ((3 - nm) == 0 or (3 - nm) >= (3 - nc)):

q.append(((nm, nc, 1 - b), path + [(m, c, b)]))

Output:



The screenshot shows a code editor with a file named 'main.py'. The code implements a breadth-first search algorithm for the Missionaries and Cannibals Problem. It starts with an initial state (3, 3, 0) and explores possible moves until it reaches the goal state (0, 0, 1). The output window displays the solution path as a sequence of states.

```
main.py  [Icons]  Share  Run  Output  Clear
```

```
9  (m, c, b), path = q.popleft()
10 if (m, c, b) in seen:
11     continue
12 seen.add((m, c, b))
13 if (m, c, b) == (0, 0, 1):
14     print("Solution Path:")
15     print(path + [(m, c, b)])
16     break
17 for x, y in moves:
18     if b == 0:
19         nm = m - x
20         nc = c - y
21     else:
22         nm = m + x
23         nc = c + y
24     if 0 <= nm <= 3 and 0 <= nc <= 3:
25         if (nm == 0 or nm >= nc) and ((3 - nm) == 0 or (3 - nm) >= (3 - nc)):
26             q.append(((nm, nc, 1 - b), path + [(m, c, b)]))
```

Output:

```
B. Deepthi Reddy
192312288
Missionaries and Cannibals Problem
Solution Path:
[(3, 3, 0), (3, 1, 1), (3, 2, 0), (3, 0, 1), (3, 1, 0), (1, 1, 1), (2,
2, 0), (0, 2, 1), (0, 3, 0), (0, 1, 1), (1, 1, 0), (0, 0, 1)]

=== Code Execution Successful ===
```